

Misinformation Detection using NLP

**DWS104 Natural Language Processing
MSc Data and Web Science
Aristotle University of Thessaloniki**

Antonis Prodromou

Orestis Georgiadis

January 18, 2026

Table of contents

1	Introduction	1
1.1	Misinformation	1
1.2	Using Natural Language Processing to tackle Misinformation	2
2	RAEmoLLM Overview	4
2.1	Shot prompting	5
2.1.1	Explicit vs Implicit Information	6
2.2	The COCO dataset	10
3	RAEmoLLM Modules	11
3.1	Index construction	11
3.1.1	Obtaining affective labels	11
3.1.2	Creating Prompt Embeddings	14
3.2	Retrieval	16
3.3	Inference	18
3.3.1	Predictions	19
4	Testing	19
4.1	Certainty score as a classification input	20
4.1.1	Background	20
4.1.2	Assessment of Certainty as a score	20
4.2	Experiment Methodology	23
4.2.1	Overview	23
4.2.2	Zero-shot model (base case)	24
4.2.3	Retrieval augmentation (RAG) with Vreg	25
4.2.4	Retrieval augmentation (RAG) with Vreg, Cert and concatenation	26

4.2.5	Retrieval augmentation (RAG) with Vreg, Cert and score averaging . .	27
4.3	Results	28
4.3.1	Zero-Shot Inference	28
4.3.2	Retrieval augmentation (RAG) with Vreg	29
4.3.3	Retrieval augmentation (RAG) with Vreg, Cert and concatenation . . .	30
4.3.4	Retrieval augmentation (RAG) with Vreg, Cert and score averaging . .	30
5	Discussion	34
5.1	Embeddings Concatenation	34
5.2	Score Averaging	35
6	Conclusions	36
	References	38

1 Introduction

This project examines RAEmoLLM, a retrieval augmented (RAG) Large Language Model (LLM) framework to address cross-domain misinformation detection using in-context learning, based on affective information. We start by outlining traditional techniques and recent developments in Natural Language Processing with regards to misinformation, move to analysing the methodology and reproducing the RAEmoLLM model, while in the third part we present the results of an experiment conducted as a variation to the method.

1.1 Misinformation

The information revolution has brought access to a variety of information instantly to the wider population, yet it has also enabled fast dissemination of both genuine and false content. The replacement of traditional media channels (e.g. newspapers, TV) by social media as the main information sources has led to the fast spread of inaccurate or misleading content. False information affects society at large while damaging the trust of the public in authentic news sources and creating suspicion or doubt to factual reporting (Lazer et al. 2023; Islam et al. 2021). For example, it was found that fabricated content spread more widely on Facebook and Twitter than accurate reporting during the 2016 U.S. presidential election (Allcott and Gentzkow 2017). To an extent, misinformation is inconsequential, but some of it has the potential to cause real-world harm, impairing public judgment, inciting panic, and causing economic losses.

Due to the massive amount of social media content, it is practically impossible to detect and flag harmful misinformation manually. Thus, conventional fact-checking can typically only counteract misinformation cases after they have gained significant traction. To provide

warnings in advance, automated systems are needed, which can be difficult to implement, as the texts that spread misinformation come from various sources (e.g. blogs, articles, social media posts) and they often transmit misinformation by relying on context and implication rather than by stating counterfactual information directly.

Further than that, Large Language Models (LLMs) are replacing traditional web searches and are therefore actively shaping what the information presented to the public is. This means that the way they have been trained is crucial to which facts are included and which not, in the answers they provide. In addition, bad actors producing mass synthetic web articles might lead to LLMs being trained on them - and thus being manipulated unintentionally.

LLMs are also good at presenting information in an authoritative way, meaning that it's difficult for the readers to easily tell when misinformation is presented. And while journalists and other sources may be provided as citations, the people who control these AI products now have a greater ability to manipulate how events are reported. Finally, one should also consider that the success of LLMs also means that as more people rely on them for information, it is more difficult for human writers and publishers to succeed - as their revenue reduces.

1.2 Using Natural Language Processing to tackle Misinformation

Misinformation detection with Natural Language Processing aims at measuring the credibility of documents, i.e., the probability a document has of not containing misinformation, using automated methods. These works have achieved significant results (Li et al. 2024; Pelrine et al. 2023). Generally, misinformation detection is a binary classification problem ("Misinformation" vs "Legitimate News") and addressed using various classification models.

Early work on misinformation detection in Natural Language Processing focused on relatively simple machine learning models combined with manually defined linguistic features. These

approaches typically relied on information such as word frequencies, writing style, sentiment polarity, or basic readability measures, which were then used by classifiers like Support Vector Machines or logistic regression (Shu et al. 2017). While they worked reasonably well on specific datasets, their main weakness quickly became apparent: models trained on one topic or platform often struggled when applied to new domains, suggesting that handcrafted features do not easily capture general properties of misinformation (Shu et al. 2017).

The move towards deep learning, and in particular transformer-based language models, led to clear improvements. Pre-trained models such as BERT made it possible to represent text in a more contextual way, which translated into better performance when these models were fine-tuned on labeled misinformation data (Devlin et al. 2019; Kaliyar, Goswami, and Narang 2021). At the same time, this progress came with practical limitations. Fine-tuning requires large annotated datasets and significant computational resources, and the resulting models often remain sensitive to changes in topic or writing style, which limits their usefulness in dynamic, real-world settings.

More recent work has therefore explored large language models in zero-shot and few-shot settings, where models are guided through prompts and example demonstrations rather than task-specific training. These approaches have been shown to generalize better across domains, especially when labeled data is scarce (Pelrine et al. 2023). In parallel, several studies have pointed out that misinformation texts tend to share several features, such as the use of emotionally charged language, often relying on fear, anger, or strong framing to influence readers Carrasco-Farré (2022); some of these features are presented in Figure 1.

Models that explicitly incorporate emotional information have shown promising results (Guo et al. 2019), but they typically still rely on supervised training. Taken together, this body of work suggests that combining the flexibility of large language models with affective cues is a

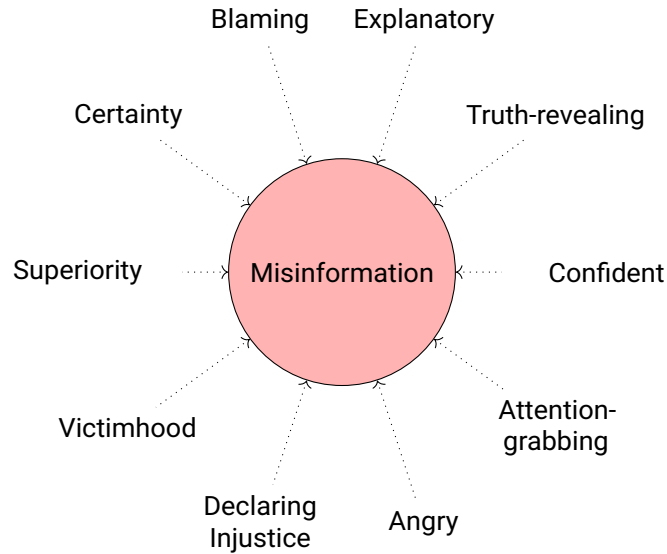


Figure 1: Characteristics of misinformation writing style and tone

promising direction for building more robust misinformation detection systems.

2 RAEmoLLM Overview

Prompts are a way to pass on instructions for a task to an LLM, providing at the same time some context to them (a “learning signal”). They can include specific demonstrations, which provide examples of the task requested. An important distinction is that - contrary to pretraining - prompting does not alter the weights of the model (e.g. through gradient descent), but aims at improving performance by changing the context and the activations. This kind of learning is called “in-context learning”.

RAEmoLLM is a Natural Language Processing framework designed to detect misinformation

across different domains by using emotional information and in-context learning. The main idea behind it is that authors of misinformation often use specific emotional language in their social media posts / article titles / texts, and these patterns remain consistent even when the topic (domain) changes. To validate this idea, the researchers conducted an affective analysis (e.g. by using t-tests) which proved that there are statistically significant affective differences between legitimate news and misinformation. If we therefore use these affective cues, we can improve the ability of Large Language Models to identify false information in new, unseen contexts.

In the RAEmoLLM framework, we do not train or fine-tune the model to perform misinformation detection. Instead, the framework is designed to use existing, pre-trained Large Language Models (LLMs) (such as Mistral-7b, ChatGPT etc.) whose performance is improved by using the aforementioned In-Context Learning (ICL) technique.

2.1 Shot prompting

In the context of RAEmoLLM, the model is provided with four specific examples to guide its inference, which is called “4-shot prompting”.

These examples are retrieved by comparing the cosine similarity between the embeddings of a) the query (i.e. the task at hand and the tweet being examined) and b) a database of tweets, framed by the same task description. Once the scores have been calculated, the tweets in the database are placed with descending order of score, and the top four examples are selected.

The authors found that cosine similarity of tweets within the same category (legit vs legit or misinformation vs misinformation) is significantly higher than similarity across different categories. They also examined how these similarity scores change as the number of retrieved examples (K) increases (from $K=4$ up to $K=64$). Conducting null-hypothesis significance testing,

they found that in the “top 4” scenario, the p-values across all affective dimensions were 0.000, which means that the top four most emotionally similar items are statistically almost certain to belong to the same veracity category. However, as K increased, the p-values began to rise, suggesting that retrieving too many examples may eventually pull in unrelated or incorrectly categorized data, which can confuse the final model.

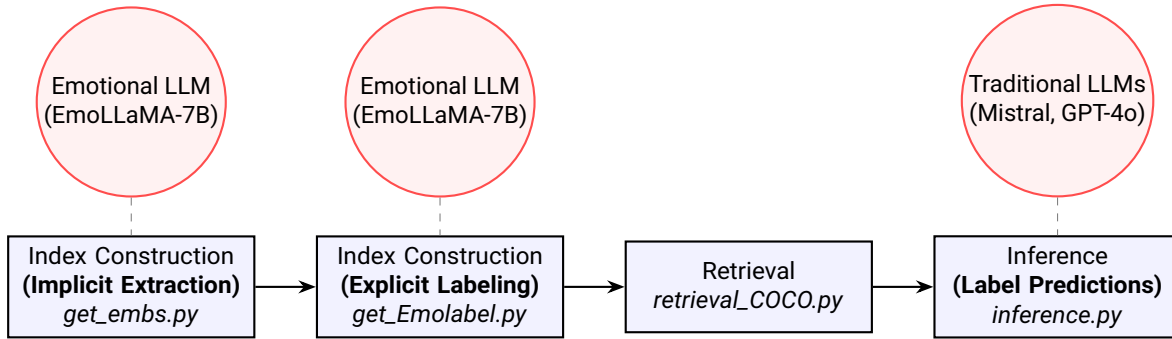


Figure 2: LLM involvement in the RAEmoLLM framework

2.1.1 Explicit vs Implicit Information

The RAEmoLLM authors examined two templates when providing prompts to the LLM: one in which the few shots are provided without their affection scores (Template 1, “Implicit” information only) and one where the affection scores were provided, too (Template 2, “Explicit” addition).

- **Zero-Shot Inference:** The base models used in the model (e.g. GPT-4o or Mistral-7b) are capable of working without few-shot prompts, a configuration referred to in the paper as “zero-shot”. In this mode, the model is given a task prompt and a target tweet but no examples to learn from. The experimental results show that while these models can make predictions, their performance is generally lower than when the RAEmoLLM framework is applied.

- **Implicit Inference:** In implicit inference, we use retrieval augmentation (RAG) to improve the predictions of the model by providing examples of tweet and label classifications in the prompt, along with the task prompt and the tweet.

These examples are selected by a) using a specially trained LLM (“Emollama-chat-7b”, see Section 3.1) which takes the tweets and the affection score calculation instructions and generates their embeddings (4096-dimensional vectors) b) generating a cosine similarity score between the query tweets and the whole tweet database and c) selecting the top k (which is 4 in our example) out of them.

The idea behind it is that by including the tokens (words) of the instructions before the tweet (e.g. “Calculate the sentiment intensity ...”), the embeddings generated by the transformer represent the tweet in the context of the specific affection (a “latent space”). Therefore similar embeddings will mean that the tweets are emotionally similar.

In other words, this method does not involve updating the model’s weights. The “training” aspect is replaced by the Retrieval-Augmented Generation (RAG) process, where the “learning” happens only within the context of the prompt provided at inference time, by providing similar tweets (in terms of affection) and whether they were labelled as ‘Conspiracy’ or not.

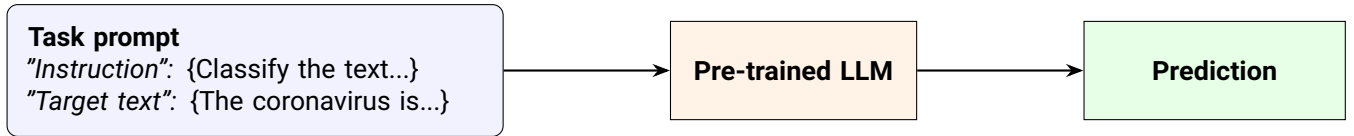
- **Explicit Inference:** In this type of inference, we include the affection scores when prompting the LLM to label the tweet as conspiracy or not, in addition to the previous instructions. The idea is that in this way the LLM is provided with a “demonstration” of how certain ranges of decimal values correlate with specific labels.

So the retrieval module still uses embeddings (implicit information) to find relevant examples, but the inference module uses explicit prompting to explain the emotional context of those examples to the model. The model is presented with a specific pattern: a text snippet, a

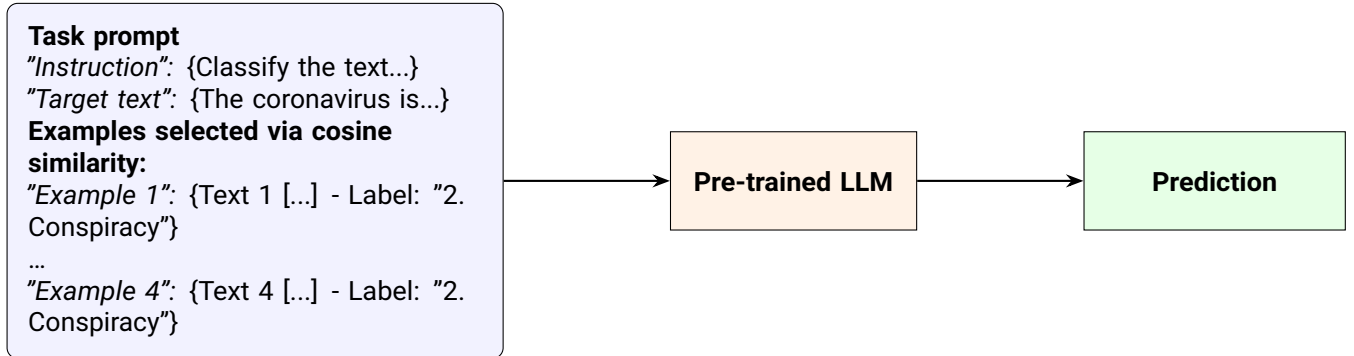
decimal value (e.g., "0.408"), and a ground-truth label (e.g., "Conspiracy"). The LLM recognizes the tokenized string as a feature that appears alongside "Fake" or "Legit" labels in the provided examples and is helped to make a better prediction.

These three types of inference are presented in Figure 3.

Zero-Shot Inference



Few-Shot, Implicit Inference



Few-Shot, Explicit Inference

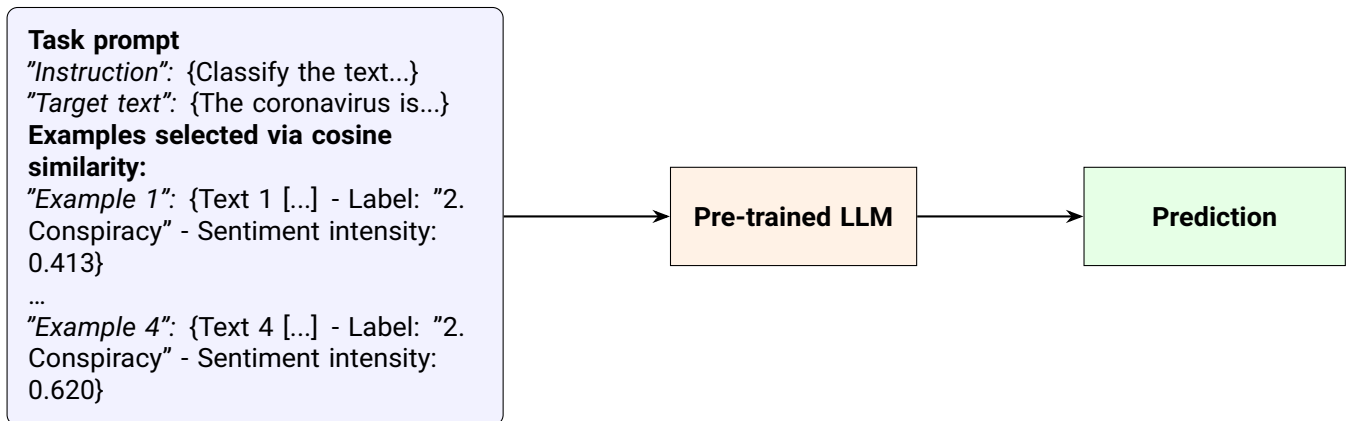


Figure 3: Zero-shot and few-shot prompting strategies for inference

The results showed that the addexpl variants (using Template 2) generally outperformed variants using Template 1.

2.2 The COCO dataset

One of the three datasets used in the paper - and the one we chose to focus on while examining the RAEmoLLM performance - is the COCO dataset (Langguth et al. 2023). It was constructed during the COVID-19 pandemic, with the primary purpose of helping to train NLP models capable of detecting stances and distinguishing topics. It consists of 3495 tweets scrapped between 17 Jan 2020 and Jun 30 2021 that were categorized manually with regards to dealing with or promoting conspiracy theories regarding the pandemic. The authors created a total of 12 categories of conspiracy theories and labeled each tweet as belonging to one of three classes in each category for a total of 41,940 labels. The three classes are:

1. **Unrelated**: The tweet is not related to that particular category.
2. **Related** (but not supporting): The tweet is related to that particular category, but does not actually promote the misinformation or conspiracy theory. Typically the authors of such tweets point out that other believe in the misinformation.
3. **Conspiracy**: The tweet is related to that particular category, and it is spreading the conspiracy theory. This requires that the author gives the impression of at least partially believing the presented ideas. This can be expressed as a statement of fact, but also in other ways such as by using suggestive questions which e.g. present the misinformation as uncertain but possible.

The dataset was provided to us for academic use, following a request to the authors through the respective [HuggingFace repository](#).

3 RAEmoLLM Modules

The RAEmoLLM system operates through three primary modules:

1. **Index construction** uses “Emollama-chat-7b”, which is an emotional LLM to generate affective embeddings.
2. **Retrieval module** finds relevant source domain examples.
3. **Inference module** uses these examples as few-shot demonstrations for classification.

The codebase includes scripts for processing datasets like PHEME, COCO, and AMTCele, along with tools to quantify sentiment intensity, valence, and specific emotion categories, which can all be grouped under the umbrella term “affection”. Given the limited resources we have for this paper, we will be using only the COCO dataset for our experiments.

3.1 Index construction

The first two steps of the module are a) obtaining affective labels and b) obtaining affective embeddings, both using [Emollama-chat-7b](#), a Large Language Model developed as part of a project aimed at comprehensive affective analysis with instruction-following capability. The model can be used for affective classification tasks (e.g. sentimental polarity or categorical emotions), and regression tasks (e.g. sentiment strength or emotion intensity) and it is finetuned based on the Meta LLaMA2-chat-7B.

3.1.1 Obtaining affective labels

We run a shell script (`get_EmoLabel.sh`) that calls 3 python scripts:

1. It first calls `construct_aff_instructs.py`, which is a script whose main method runs 5 functions; each reads in the tweets' `*.csv` file and generates a JSON file which pairs each tweet with the instructions that will be given to Emollama-chat-7b in order to assign 5 different affection scores to them:
 - El-reg Emotion Intensity: a continuous value ranging from 0 to 1 for four emotions ["anger", "fear", "joy", "sadness"].
 - El-oc: Emotion Intensity: an ordinal value rating the intensity of emotion of the tweeter, taking values 0 (can't be inferred), 1 (low amount of emotion can be inferred), 2 (moderate amount of emotion) and 3 (high amount of emotion), again for the four emotions mentioned.
 - V-reg: Detecting Valence or Sentiment Intensity, being a continuous value ranging from 0 to 1.
 - V-oc: Detecting Valence using the ordinal equivalent and taking values [{"3": "very positive mental state", "2": "moderately positive mental state", "1": "slightly positive mental state", "0": "neutral or mixed mental state", "-1": "slightly negative mental state", "-2": "moderately negative mental state can be inferred", "-3": "very negative mental state"}].
 - Task E-c: Detecting Emotions (multi-label classification), taking [anger, anticipation, disgust, fear, joy, love, optimism, pessimism, sadness, surprise, trust].

Given the scope of our project, we will be examining only "V-reg" from the above RAEmoLLM scores in our experiments (this choice is explained in Section 4.3).

The above functions collect the instructions in a dataframe (with no score assigned, as it hasn't been calculated yet), which is then exported as `*-affective-all.json`.

2. The shell script then calls `get_EmoLabel.py`, which calculates (assigns) the aforementioned affection scores to all the tweets. It starts by reading in the `*.json` file

of the previous step with each tweet and the respective [5] instructions and converts it into a list. It then applies a tokenizer that converts text into integer numbers (tokens). The tokenizer uses left padding for shorter tweets, because of the way the next tokens are generated in decoders: the last token is the one whose logits are used for next-token prediction, so it wouldn't make sense it being a special padding token.

The script uses PyTorch to take in batches of the list and Emollama-chat-7b and generate the outputs, i.e. to assign the 5 different affection scores to the tweets, as those were described earlier. The padding tokens used earlier are filtered using an `attention_mask`, that takes in the batched inputs and returns a binary mask of the same shape, where "1" denotes a token that needs to be attended to and "0" a padding token. The mask is passed as an argument to `model.generate()` and adds $-\infty$ to attention scores for padding tokens, as is described in the below formula:

$$\text{head} = \text{Softmax} \left(\text{Mask} \left(\frac{QK^T}{\sqrt{d_k}} \right) \right) V$$

This ensures that no token attends to the padding token. The script returns a `*.predict.json` file containing the affection scores that the model estimated for affection types and all the tweets (Figure 4).

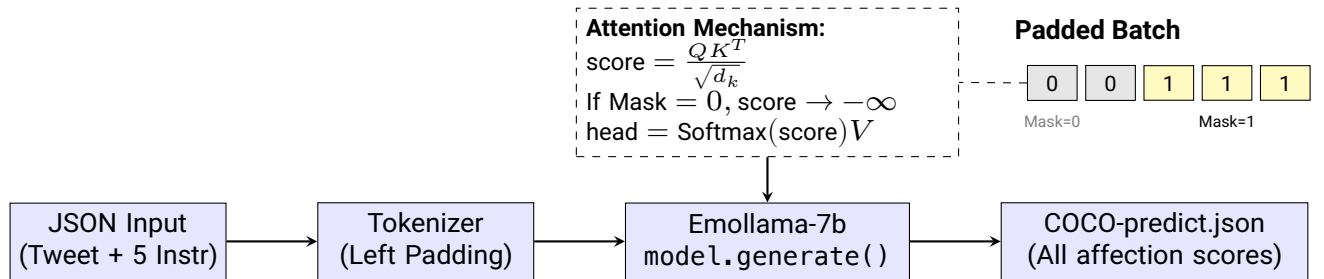


Figure 4: The process for calculating affection scores in RAEmoLLM.

3. We then call `postprocess_label.py`, which cleans the results and merges them back

into a structured format (CSV) for use in the rest of the pipeline. More specifically, it imports the combined `*.predict.json` file we just generated and does the following:

- a. It takes both the original tweets dataset and the generated labels, and removes the prompt we gave.
- b. Breaks them down into the labels applied.
- c. Turns them into dataframes and returns a `*.csv` file where columns with the generated affection scores have been added to the collection of the original tweets and conspiracy sector labels (`COCO-add-aff.csv`).

3.1.2 Creating Prompt Embeddings

The second shell executed is `get_embs.sh`, which generates the embeddings for the dataset and the affection score we specify. Due to time and resource limitations (GPU process time and cost), for this project we use the COCO dataset and the Vreg score (Valence or Sentiment Intensity ranging from 0 to 1). The only script it executes is `get_embs.py`, which does the following:

- Reads in the csv file for a specific attention score (e.g. `COCO-Vreg.csv`), whose rows at this stage have the following form:

Input for the generation of embeddings

Task: Calculate the sentiment intensity [...]

Tweet: The coronavirus seems suspicious [...]

Intensity Score: [...]

i.e. each row contains the instruction, the tweet and the specified affection rating. It transforms them into list items, each of which contains both the tweet and the instruction, e.g. ["Human: \n[Instruction & Tweet]\n\nAssistant:\n"].

- Once we have configured the tokenizer, we take each prompt and we run a transformer (a LlamaModel 3-8b) to convert it into numerical tensors. According to its [online documentation](#), this Llama model has a 4096 vector embedding size. We set `output_hidden_states=True` to get the hidden representations from all transformer layers. The last hidden state encodes high-level semantic features (including intent or emotion), due to the successive self-attention transformations across the layers that precede it. Earlier layers usually contain information related more to the structure or relationships between the words in the text.

We then average these last hidden states of all the words in the tweet, and create a single vector (embedding) that represents that whole specific tweet and the instruction that are used to generate the affection score, without the score, as is depicted in Figure 5. In this way, we incorporate implicit representation in the embedding, as was explained earlier in Section 2.1.1.

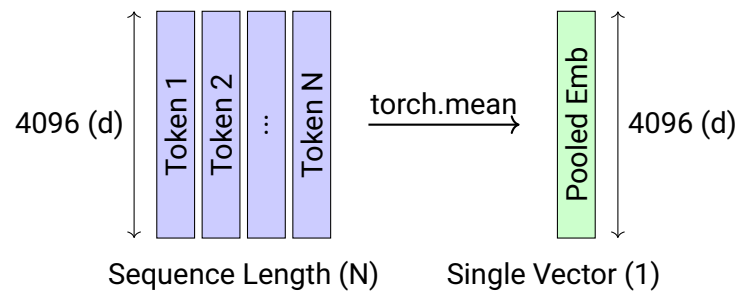


Figure 5: Creating single embeddings for each Tweet. The sequence embedded contains both the tweet and the prompt for generating affection scores.

The script outputs a `.pth` (PyTorch) file containing the embeddings for a specific affection

score (e.g. COCO-Vreg-embeddings.pth).

3.2 Retrieval

The retrieval stage of RAEmoLLM aims at recommending the most relevant examples from a “source domain” to help “Emollama-chat-7b” classify misinformation contained in an unseen tweet (the new “target domain”). It is the middle stage of the process between data indexing and final model inference and it essentially creates a few-shot prompt to improve our predictions.

More specifically, we run a shell (`retrieval.sh`) that takes the embeddings we just created (at the Index stage) and does the following:

- The `get_answer()` method takes each tweet (row) and filters the labels (conspiracy sectors / topics) most affiliated with the tweet, which have been set by the authors as those scored with 2 or 3. This creates a new column in the dataframe of the tweets, called ‘topic’, where each tweet has been assigned its topics e.g. `dataframe['topic']=(4, 10)`.
- We then run `process_VregVocEc()` that takes the dataframe we created with the tweets, and translates it into a Python list of dictionaries that has the following structure:

```
datas = [  
    # each tweet has 5 keys  
    {  
        "domain": ...,  
        "label": ...,  
        "content": ...,  
        "aff_info": ...,
```

```

        "embedding": ...
    },
    ...
]

```

Where the keys stand for the following:

- **“domain”**: the conspiracy topic e.g. “1, 4” or “non-conspiracy”.
- **“label”**: the classification of the tweet, e.g. “1. Related (but not supporting)”.
- **“content”**: the raw text of the post/tweet.
- **“aff_info”**: the Affective information score.
- **“embedding”**: the vector representing the semantic meaning of the text.

Finally, for each of the dictionaries (tweets), we run `get_retri()`, which calculates cosine similarity (using `tec2vec semantic_search`) to find the 4 tweets (examples) that have the highest similarity scores in the $[0,1]$ range. The retrieved examples include both the tweet and its verified label (e.g., “Conspiracy” or “Unrelated”).

The final outcome of the retrieval process is a `.pth` (PyTorch) file that now contains the tweet, its label, and a list of the 4 most similar tweets found in the database, which will be used as 4-shot prompts.

3.3 Inference

The inference stage starts with constructing the instructions for inference. The authors followed the EmoLLM guidelines (Liu et al. 2024) to define the prompt that will be used to ask the LLM to do the classification. For the COCO dataset, the zero-shot prompt ('Base Case') it is the following:

The zero-shot prompt

Classify the text regarding COVID-19 conspiracy theories or misinformation into one of the following three classes: 0. Unrelated. 1. Related (but not supporting). 2. Conspiracy (related and supporting).

The authors explain that the purpose of keeping the "0. Unrelated" classification is to test the robustness of the LLM by increasing the complexity of the task.

The `construct_instructs_*` method loads the test dataset and provides the dictionary for each tweet obtained in the retrieval stage earlier (the `*.pth` file) to the LLM. In the case of RAG implicit prompting, the "Instruction" column contains a) the task prompt ("Classify the...") b) the retrieved examples (4 few-shot prompt demonstrations) with their labels (e.g. "Conspiracy") and c) the target text (query).

The script saves the instructions initially as a json file (`test.json`) where each line is a dictionary containing the instruction and the ground-truth answer, as a csv (`test.csv`) and back to json (`COCO-Vreg-4096d.json`). This pattern (JSON to CSV to JSON) is done to ensure our data has the correct format and data types ('data sanitization'), as by loading into a pandas dataframe, we flatten it and "force" the correct format.

3.3.1 Predictions

We then run the actual inference script, to get the predictions (`inference.py`). We use the Hugging Face transformers library to load the transformer (`Mistral-7B-Instruct-v0.2` in our example), read in the `*.json` file of the previous step, tokenize (padding left once again), and conduct batch inference. The prediction contains the tweet, the prompt, the few-shot examples, the output of the model and an explanation, in the following form:

RAEmoLLM output

According to the above information, the label of target text:

[prediction]

e.g. 2. Conspiracy (related and supporting)

[explanation]

(e.g. The text expresses the belief that Bill Gates and his wife were responsible for the creation of the coronavirus, which is a conspiracy theory.)

The output is saved in `*.json` format (`C0C0-Vreg-4096d-predict.json`).

4 Testing

We chose to run four experiments as part of our report, in order to a) reproduce some of the findings of the original paper and b) attempt to extend the RAEmoLLM framework to also account for a new score, the certainty degree ('Cert') expressed in each tweet, taking continuous values between [0,1].

4.1 Certainty score as a classification input

4.1.1 Background

Certainty refers to a sense of conviction or confidence that characterizes language. Previous research suggests that certainty in language increases consumer engagement to brands' social media messages (Pezzuti, Leonhardt, and Warren 2021) and is used in politics to affect voters' preferences (Hart and Childers 2004). We therefore want to examine whether a) certainty is more prevalent in misinformation texts compared to legitimate news, and whether b) including it as a parameter provided to an LLM tasked with misinformation predictions can improve its performance.

4.1.2 Assessment of Certainty as a score

To validate our idea, we first need to establish whether the certainty of a text - here represented by a certainty score ("Cert") - differs depending on whether a text is labeled as Misinformation ("Conspiracy" for the COCO dataset) or not. The steps that we followed were:

1. We used the same EmoLLM guidelines (Liu et al. (2024)) that the authors used in the original RAEmoLLM paper to define the prompt that will be used to ask the LLM to assign the certainty scores. The prompt is:

Prompt to extract the certainty score

Calculate the certainty degree score of the below tweets, which should be a real number between 0 (extremely uncertain) and 1 (extremely certain).

2. We then ran the EmoLLM and assigned a "Certainty" score to each tweet at the index construction stage of the experiment (`get_EmoLabel.py`).

3. Finally, we defined the question in statistical terms. These are the following:

- We have two samples (those labeled as “Conspiracy” vs the rest), which are independent to each other (the sets of tweets have been randomly chosen).
- We are examining a continuous variable (“Cert”), so we can compare its mean between the two samples (μ_1 and μ_2 , with standard deviations σ_1 and σ_2).
- We will conduct a two-side test, with the Null Hypothesis being $H_0: \mu_1 = \mu_2$ that there is no significant difference in the certainty score means between the two samples. The Alternative Hypothesis $H_0: \mu_1 \neq \mu_2$ is that there is a statistically significant difference.
- Based on the above, we will use a Welch’s t-test for the evaluation, as a safer and simpler choice that does not assume equal variances between the two groups.

We ran the index construction stage of the RAEmoLLM framework and the charts in Figure 6 and Figure 7 show that the scores are not normally distributed in the samples.

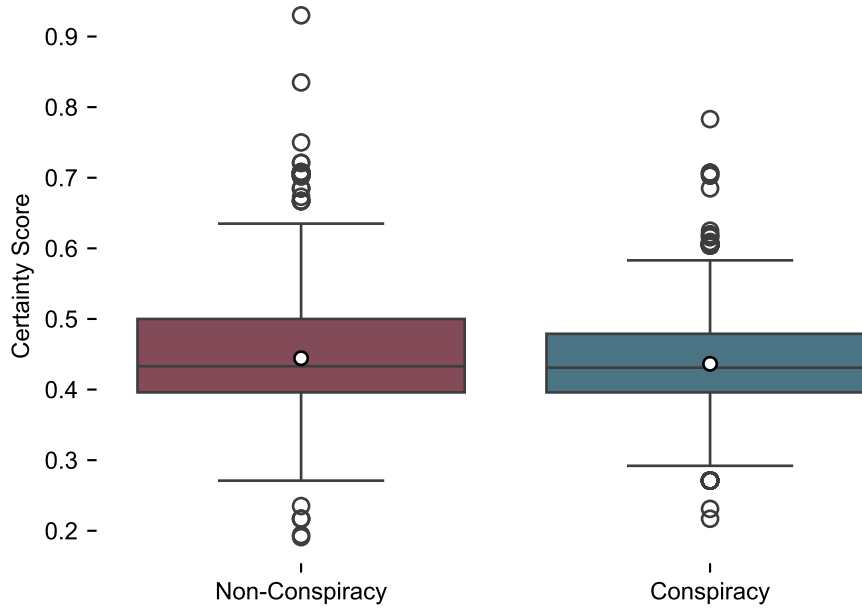


Figure 6: Box plot of Certainty scores distributions, Conspiracy vs Non-Conspiracy tweets

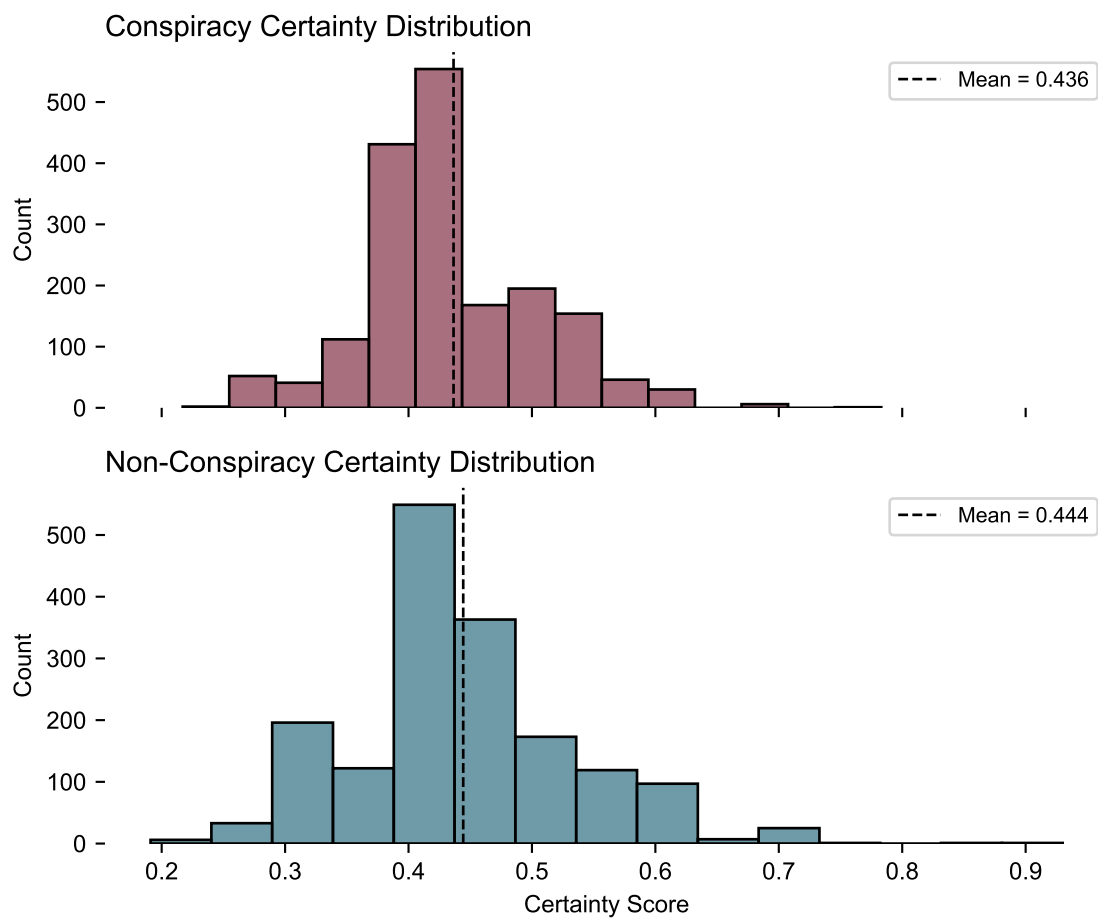


Figure 7: Histograms of Certainty scores distributions, Conspiracy vs Non-Conspiracy tweets

This is confirmed by the results of the Welch’s t-test presented in Table 1.

Table 1: Welch’s two-sample t-test comparing certainty scores between conspiracy and non-conspiracy tweets

	Statistic	Value
0	t-statistic	-2.8730
1	p-value	0.0041

However, while our initial assumption was right in that there is a difference in the certainty score between misinformation and legitimate texts, it was wrong in that misinformation shows more certainty in the opinions expressed. The mean Conspiracy group certainty score is lower than the mean of the Non-Conspiracy group, with a t-statistic = -2.8730. In addition, given that the p-value $0.0041 < 0.05$, based on the evidence we have we can reject the Null Hypothesis, and we therefore accept that at a 0.05 significance level there is a difference between legitimate news and misinformation with regards to the certainty of the tweets, with legitimate news showing more certainty than misinformation tweets.

4.2 Experiment Methodology

4.2.1 Overview

These models we ran were the following:

1. **Zero-shot model (base case).**
2. **Retrieval augmentation (RAG) with Vreg:** In this case we use as explicit information the Valence or Sentiment Intensity (Vreg) of the tweets (a continuous value ranging from 0 to 1). This was chosen because the paper uses Vreg as the primary metric for its overall

performance results, ablation studies, and comparisons across different LLMs. Most of the baseline tables in the paper are built using Vreg instructions.

3. **Retrieval augmentation (RAG) with Vreg, Cert and concatenation:** Here we use as explicit information the Valence or Sentiment Intensity of the tweets (a continuous value ranging from 0 to 1) in addition to explicit information on the Certainty degree of the tweets (also a continuous value ranging from 0 to 1). The two embeddings are concatenated to form a 8192-dimension embedding (more details are provided in Section [4.2.4](#) below).
4. **Retrieval augmentation (RAG) with Vreg, Cert and score averaging:** Similar to before, we use as explicit information the Valence or Sentiment Intensity of the tweets in addition to explicit information on the Certainty degree of the tweets. However, in this case we keep embeddings separate, calculate two scores (i.e. scalars) and combine to retrieve the RAG examples (Section [4.2.5](#)).

The experiments were run on the COCO dataset. The models were evaluated using the same leave-one domain-out strategy described in the paper, which consists in sequentially selecting a specific domain as the test set and the remaining domains as the training set. The authors use the “Fake Virus”, “Harmful Radiation”, and “Depopulation” topics as the test set, and the other topics as the retrieval dataset.

Finally, the experiments were run on Google Colab T4 GPU notebooks, with a runtime of approximately 6 hours each.

4.2.2 Zero-shot model (base case)

The zero-shot model (base case) represents the standard performance of a Large Language Model when it is tasked with misinformation detection without any few-shot examples or retrieval augmentation. This involves presenting the model with a target tweet from the COCO

dataset and a specific instruction to classify it into one of three categories: Unrelated, Related (but not supporting), or Conspiracy. Unlike the RAEmoLLM framework, the base case does not use implicit affective embeddings or explicit sentiment intensity scores during inference to assist in the classification process.

4.2.3 Retrieval augmentation (RAG) with Vreg

The explicit Retrieval-Augmented Generation (RAG) approach for Vreg (Sentiment Strength) on the COCO dataset is the most advanced configuration of the RAEmoLLM framework. Unlike the zero-shot base case, this method uses a retrieval module to provide the LLM with context combining implicit data and explicit data:

- **Implicit Retrieval:** The system uses 4096-dimensional affective embeddings to identify the top-4 most similar examples from source domains.
- **Explicit Data:** For the target tweet and each of the four retrieved examples, the system also presents the Vreg score, a decimal between 0 (extremely negative) and 1 (extremely positive), which was generated during the index construction stage.

The final prompt includes the task, the target tweet accompanied by its specific Vreg score and four retrieved examples presented in the following format:

Prompt used for Retrieval Augmentation (RAG) with Vreg

Text: *The coronavirus seems suspicious [...]*

Sentiment intensity: *0.606*

The label of this text: *Conspiracy*

4.2.4 Retrieval augmentation (RAG) with Vreg, Cert and concatenation

Building on the aforementioned explicit RAG approach, we calculate separate embeddings for 'Vreg' and 'Cert' (which stands for Certainty), following the same procedure described in Section 3.1.2. Hence we read in the tweets and then take each prompt (i.e. both the tweet and the instruction), and run a transformer (a LlamaModel) to convert it into numerical tensors ($n = 4096$).

The result is two embeddings, one for each score:

- `COCO-Vreg-embeddings.pth` for the tweet's emotion valence
- `COCO-Cert-embeddings.pth` for the tweet's certainty

Once the embeddings have been calculated, we concatenate them at the index construction level, using a new `average_embeddings.py` function.

$$emb_{VregCert} = [emb_{Vreg}; emb_{Cert}]$$

The idea behind concatenation is that since each embedding captures different information (emotion valence vs certainty), we want to avoid averaging or summing the embeddings, as that might lead to losing information (one cancelling the effects of the other) or creating useless information (noise). This means that the length of the embeddings doubles (from 4096 to 8192), which comes with a) increased computation costs and b) the risk of overfitting on the training data, as essentially we are doubling the number of features!

The new averaged embedding is then added to the pipeline, following the exact same steps as in the original model.

4.2.5 Retrieval augmentation (RAG) with Vreg, Cert and score averaging

In this case, we keep the 4096-length embeddings separate, compute cosine similarity within each space and get two `semantic_search` scores (i.e. scalars), one for valence and one for certainty. We combine these scores equally, using $\alpha = 0.5$ as a weight for each. The few shot tweets then retrieved are the top 4 with regards to this score:

$$\begin{aligned} \text{score} = & \alpha \cdot \cos(\text{tweet}_{\text{target}}(Vreg), \text{tweet}_{\text{retrieval}}(Vreg)) \\ & + (1 - \alpha) \cdot \cos(\text{tweet}_{\text{target}}(Cert), \text{tweet}_{\text{retrieval}}(Cert)) \end{aligned}$$

where $\alpha = 0.5$.

In other words, the affection score generation and the inference logic remain the same; what changes is the way the few-shot examples are retrieved (i.e. the `retrieval_COCO.py` code).

This method was chosen to a) avoid increasing the dimensions of the vectors and the resulting overfitting on the training data b) avoid cancelling out information in the embeddings by averaging them and c) to allow a flexible way of further experimenting by applying separate weighting to the two scores.

4.3 Results

4.3.1 Zero-Shot Inference

Running the base model on the dataset, we got the following results:

Table 2: Zero Shot Model Classification Performance (with “Unrelated” distinction)

Metric	Precision	Recall	F1-Score	Support
Unrelated	0.60	0.04	0.07	906
Related	0.13	0.36	0.20	248
Conspiracy	0.35	0.59	0.44	612
Accuracy	—	—	0.28	1766
Macro Avg	0.36	0.33	0.24	1766
Weighted Avg	0.45	0.28	0.22	1766

Comparing the results to Table 3 of the paper, the “Conspiracy” metrics from our reproduction run are close to the ones obtained by the authors of the paper, e.g. the F1-score calculated in our experiment is 0.4407 vs 0.4673 provided in the paper.

We assume that the “Conspiracy” metrics are only being evaluated against the “Non-Conspiracy” category, as the authors state that the “Unrelated” category (and likely the classification compared to “Related”) was only included to test the robustness of the LLM. Applying this rational gives the below classification metrics:

Table 3: Zero-shot model - Classification performance (“Conspiracy” vs “Non-Conspiracy”)

Metric	Precision	Recall	F1-Score	Support
Non-Conspiracy	0.6584	0.4142	0.5085	1154
Conspiracy	0.3500	0.5948	0.4407	612
Accuracy	—	—	0.4768	1766
Macro Avg	0.5042	0.5045	0.4746	1766
Weighted Avg	0.5515	0.4768	0.4850	1766

4.3.2 Retrieval augmentation (RAG) with Vreg

Table 4: RAG RAEmoLLM with explicit Vreg - Classification performance

Metric	Precision	Recall	F1-Score	Support
Non-Conspiracy	0.8472	0.8934	0.8697	1154
Conspiracy	0.7760	0.6961	0.7339	612
Accuracy	—	—	0.8250	1766
Macro Avg	0.8116	0.7947	0.8018	1766
Weighted Avg	0.8225	0.8250	0.8226	1766

The results show that the performance of the RAG model (with Vreg) is significantly better across all metrics compared to the base zero-shot model. The overall accuracy increased from 47.68% (base) to 82.50% (RAG); the F1-Score increased from 47.46% to 80.18%, indicating that

the RAG model is far more balanced across both classes. The RAG model is also significantly better at identifying non-conspiracy (legitimate) content, with Recall rising from 0.41 to 0.89. Finally, the precision in predicting conspiracy tweets more than doubled, from 0.35 (meaning the base model flagged far too many false positives) to 0.77 using Vreg RAG.

4.3.3 Retrieval augmentation (RAG) with Vreg, Cert and concatenation

Table 5: RAG RAEmoLLM with explicit Vreg, Cert (Concatenation) - Classification performance

Metric	Precision	Recall	F1-Score	Support
Non-Conspiracy	0.8347	0.9055	0.8687	1154
Conspiracy	0.7879	0.6618	0.7194	612
Accuracy	—	—	0.8211	1766
Macro Avg	0.8113	0.7837	0.7940	1766
Weighted Avg	0.8185	0.8211	0.8169	1766

Comparing the RAG with explicit Vreg model to the Concatenation version (which adds “Cert” or certainty information), the former performs slightly better overall (Accuracy: 0.8250 (Vreg) vs. 0.8211 (Concatenation), F1-Score: 0.8018 (Vreg) vs. 0.7940 (Concatenation) and Conspiracy Recall of 0.696 vs 0.661). While the Concatenation method slightly improves precision for the minority class, it seems that adding the “Cert” (certainty) data via concatenation introduces a slight “noise” that slightly worsens the model’s effectiveness.

4.3.4 Retrieval augmentation (RAG) with Vreg, Cert and score averaging

Table 6: RAG RAEmoLLM with explicit Vreg, Cert (Score averaging) - Classification performance

Metric	Precision	Recall	F1-Score	Support
Non-Conspiracy	0.8832	0.8518	0.8672	1154
Conspiracy	0.7381	0.7876	0.7621	612
Accuracy	—	—	0.8296	1766
Macro Avg	0.8107	0.8197	0.8146	1766
Weighted Avg	0.8329	0.8296	0.8308	1766

The results show that the Score Averaging model (using both Vreg and Cert) performs better than all the previous versions, as it achieves the highest overall Accuracy (0.8296) and Weighted F1-Score (0.8308). The most significant improvement in this model is its ability to identify conspiracy content, as Recall for “Conspiracy” rose to 0.7876, which is a great improvement over both the Explicit Vreg (0.6961) and the Concatenation (0.6618) models.

Choosing evaluation metrics

The test dataset we used in this evaluation has a partially uneven distribution. In these cases, the Accuracy metric is unreliable because it does not take into account the initial distribution of positive and negative samples. For example, if in our case we set all predictions equal to the value of the greater group ‘Non-Conspiracy’, we would get an accuracy of 65.3% by correctly predicting only the cases of ‘Non-Conspiracy’, without predicting a single case of ‘Conspiracy’. In this case, the model has no predictive value, and this accuracy is called ‘Baseline Accuracy’.

Instead, it is more appropriate to use Recall and F-score, for the following reasons:

- Recall is equal to $\text{True Positives} / (\text{True Positives} + \text{False Negatives})$, which means that the fewer Conspiracy tweets are misclassified as 'Non-Conspiracy', the higher it is.
- F-score: incorporates both Recall and Precision ($\text{True Positives} / (\text{True Positives} + \text{False Positives})$), thus offering a balance in assessing the performance of models for predicting both classes.

Also, until recently, in cases where posts on social media were marked as misinformation, human review usually followed to verify the result. If it was done by mistake, the label was removed and the tweet was made public. The cost in this case is the inconvenience felt by the user and, of course, the increased cost for the staff performing the verifications.

However, if a tweet containing misinformation is not flagged appropriately (low recall), then no review takes place and it is freely disseminated.

The F1-score comparative chart for all four models is presented in Figure 8. The best model according to the results is RAG using explicit valence and certainty prompting ('Vreg + Cert (Score Averaging)') with an F1 of 0.7621.

The recall values for the "Conspiracy" label are given in Figure 9.

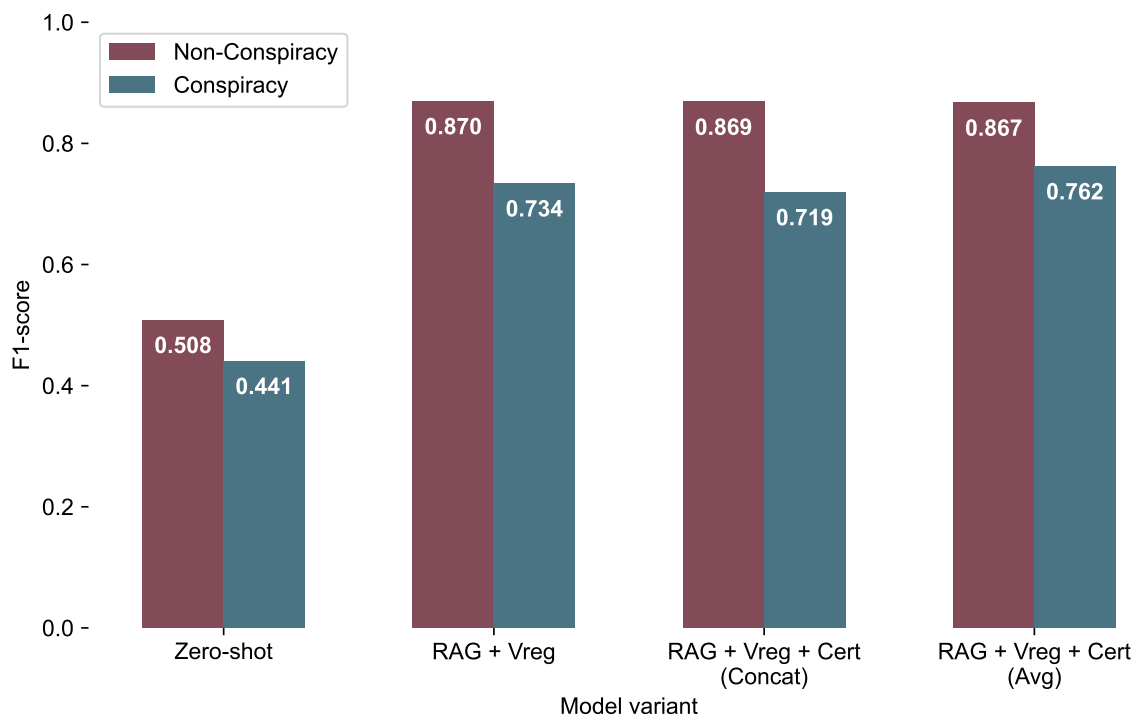


Figure 8: Classification F1 score for all four models

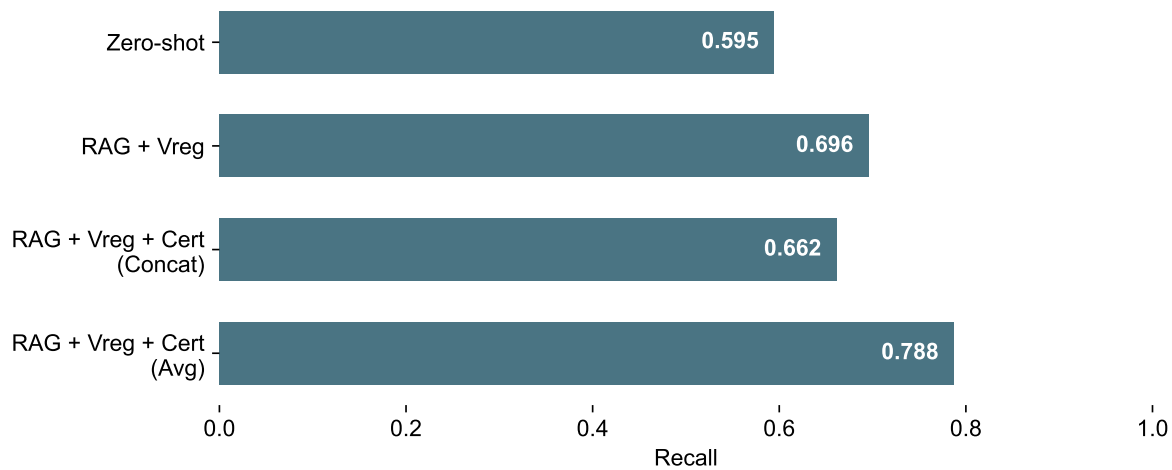


Figure 9: Recall score for “Conspiracy”, for all four models

5 Discussion

Retrieval-Augmented Generation (RAG) improves the performance of transformers by enhancing the model from a system that relies solely on its internal, static parameters into a dynamic model that can reference external evidence provided at runtime. Instead of retrieving a fact based on weights set during pretraining from a huge database, with RAG the transformer generates an answer where the next token probabilities are generated based on the attention mechanism on the retrieved passages $R(q)$, the task prompt, and the previously generated tokens:

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i \mid R(q); \text{prompt}; q; x_{<i})$$

5.1 Embeddings Concatenation

In the embeddings concatenation model, increasing the model dimensionality (d) has a quadratic effect on the total number of parameters in the network. This is because the number of non-embedding parameters N in a transformer can be approximated using the following formula:

$$N \approx 12 \cdot n_{\text{layer}} \cdot d^2$$

Where n_{layer} is the number of stacked transformer blocks and d is the input and output dimensionality (embedding size). By doubling d from 4096 to 8192, the d^2 term increases by a factor of four. This results in increased computation costs and significantly higher hardware requirements for both training and inference.

The embedding dimension d affects every layer within the transformer block:

- In the Multi-head Attention layer, the weight matrices for queries (W_Q), keys (W_K), and values (W_V) have shapes analogous to d ($[d \times d_k]$). Doubling d increases the size of these projection matrices.
- The Feedforward Network (FFN) layer involves a hidden layer d_{ff} that is usually larger than d (e.g., $d_{ff} = 4d$). The formula for the FFN calculation is:

$$FFN(x_i) = \text{ReLU}(x_i W_1 + b_1) W_2 + b_2$$

Therefore, as x_i grows from 4096 to 8192, the weight matrices W_1 and W_2 must scale accordingly to handle the larger input vectors.

- Layer normalization is a variation of the z-score from statistics, applied to the embedding vector of a single token in a hidden layer. Thus the input to layer norm is a single vector of dimensionality d and increasing it also increases the number of elements processed in each normalization step.

5.2 Score Averaging

The success of the Score Averaging approach in the RAEmoLLM framework is explained by its ability to integrate two affective “spaces”. For each individual affective dimension (such as V_{reg} for valence or $Cert$ for certainty), the system calculates the cosine similarity between the target tweet vector ($\mathbf{v}$) and a source domain vector ($\mathbf{w}$):

$$\cos(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{\|\mathbf{v}\| \|\mathbf{w}\|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

The explanation of its success can be explained by a simple example: if we applied an additional filter to an otherwise equally distributed probability of classification, the slight but statistically significant difference (found when running the Welch’s t-test, see Table 1) in the certainty expressed in non-conspiracy tweets, will marginally but with an actual effect, correctly classify 1% more tweets.

The Score Averaging method increases computational costs compared to the single-attribute version (e.g. using only Vreg), but it does so in a linear way rather than quadratic, as was the case with embeddings concatenation. At the indexing stage, we must run the `get_embs.py` script twice to generate two separate 4096-dimensional vectors for every tweet. Similarly, we must store two `.pth` files instead of one, doubling the storage requirements for the retrieval database; and finally the system must perform two separate cosine similarity calculations and then combine them at the retrieval stage.

It must be noted that the inference module remains the same size in both cases, as it still processes the same number of tokens in the prompt.

6 Conclusions

In this project, we successfully reproduced the main findings of the original RAEmoLLM framework paper on the COCO dataset, confirming that affective RAG significantly outperforms zero-shot models. The zero-shot base case performed poorly, achieving an F1-score of 0.4407, but applying the standard RAEmoLLM (RAG with Vreg) increased the F1-score to 0.7339, proving that providing the LLM with emotionally similar examples is highly effective for cross-domain detection.

The “certainty” expressed in a tweet proved to have a statistically significant difference (p-value

0.0041) in the certainty levels between legitimate news and conspiracy tweets. However, it was found that legitimate news show higher certainty than conspiracy tweets, despite the contrary initial assumption. This could be due to the fact that conspiracy theories might use “suggestive questions” or present ideas as “uncertain but possible”.

We explored two ways to combine multiple affective dimensions, using Valence intensity and Certainty, and concluded that “Score Averaging” shows the best performance between the given models. This method achieved the highest performance (F1-score: 0.7621) by keeping embeddings separate and combining their similarity scores. It avoids the risk of “cancelling out” information and allows for flexible weighting of different emotions.

Concatenation of Valence and Certainty embeddings performed worse (F1-score: 0.7194) than simply providing Valence scores (F1: 0.7339). This could be due to the fact that doubling the vector dimensions (from 4096 to 8192) increases overfitting due to the higher number of features.

Future work could experiment with different weighting for the affective scores used during retrieval to find a better balance between valence, certainty, and other emotions. One could also develop specific prompts or affective dimensions to identify “suggestive” tones (rather than “certainty” ones) that are used to manipulate readers without making direct claims.

We conclude that the RAEmoLLM framework is particularly effective at emotional detection. Even when conspiracy theories use subtle language or implications rather than outright lies, the “affective nature” of the language used allows LLMs to categorize them correctly when provided with the right few-shot demonstrations.

References

- Allcott, Hunt, and Matthew Gentzkow. 2017. "Social Media and Fake News in the 2016 Election." *Journal of Economic Perspectives* 31 (2): 211–36.
- Carrasco-Farré, Carlos. 2022. "The Fingerprints of Misinformation: How Deceptive Content Differs from Reliable Sources in Terms of Cognitive Effort and Appeal to Emotions." *Humanities and Social Sciences Communications* 9 (1): 162. <https://doi.org/10.1057/s41599-022-01174-9>.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. "BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding." In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 4171–86. Association for Computational Linguistics.
- Frisli, Siri. 2025. "Language, Power, and Misinformation: A Mixed-Method Analysis of COVID-19 Discourses on Norwegian Twitter." *Social Media + Society* 11 (2): 20563051251347614. <https://doi.org/10.1177/20563051251347614>.
- Guo, Han, Juan Cao, Yufei Zhang, Jintao Guo, and Xirong Li. 2019. "Rumor Detection with Hierarchical Social Attention Network." *Proceedings of the 27th ACM International Conference on Multimedia*, 943–51.
- Hart, Roderick P., and Jay P. Childers. 2004. "Verbal Certainty in American Politics: An Overview and Extension." *Presidential Studies Quarterly* 34 (3): 516–35. <http://www.jstor.org/stable/27552611>.
- Islam, Md Saiful, Abu-Hena Mostofa Kamal, Alamgir Kabir, Dorothy L. Southern, Sazzad Hossain Khan, S. M. Murshid Hasan, Tonmoy Sarkar, et al. 2021. "COVID-19 Vaccine Rumors and Conspiracy Theories: The Need for Cognitive Inoculation Against Misinformation to Improve Vaccine Adherence." *PLOS ONE* 16 (5): e0251605.

- Kaliyar, Rohit Kumar, Anurag Goswami, and Pratik Narang. 2021. "FakeBERT: Fake News Detection in Social Media with a BERT-Based Deep Learning Approach." *IEEE Transactions on Computational Social Systems* 8 (4): 932–42.
- Langguth, Johannes, David Thulke Schroeder, Petra Filkuková, Stefan Brenner, Joshua Phillips, and Konstantin Pogorelov. 2023. "COCO: An Annotated Twitter Dataset of COVID-19 Conspiracy Theories." *Journal of Computational Social Science*, April, 1–42. <https://doi.org/10.1007/s42001-023-00200-3>.
- Lazer, David M. J., Matthew A. Baum, Yochai Benkler, Adam J. Berinsky, Kelly M. Greenhill, Filippo Menczer, Miriam J. Metzger, et al. 2023. "The Science of Fake News." *CoRR* abs/2307.07903.
- Li, Minghao, Weizhi Wang, Fuli Feng, Fei Zhu, Qiang Wang, and Tat-Seng Chua. 2024. "Think Twice Before Trusting: Self-Detection for Large Language Models Through Comprehensive Answer Reflection." In *Findings of the Association for Computational Linguistics: EMNLP 2024*, edited by Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, 11858–75. Association for Computational Linguistics.
- Liu, Zhiwei, Kailai Yang, Qianqian Xie, Tianlin Zhang, and Sophia Ananiadou. 2024. "EmoLLMs: A Series of Emotional Large Language Models and Annotation Tools for Comprehensive Affective Analysis." In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5487–96. KDD '24. ACM. <https://doi.org/10.1145/3637528.3671552>.
- Pelrine, Kevin, Asma Imouza, Christian Thibault, Mikhail Reksoprodjo, Chaitanya Gupta, Jonas Christoph, Jean-François Godbout, and Reihaneh Rabbany. 2023. "Towards Reliable Misinformation Mitigation: Generalization, Uncertainty, and GPT-4." In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6399–429. Association for Computational Linguistics.

- Pezzuti, Todd, James M. Leonhardt, and Caleb Warren. 2021. "Certainty in Language Increases Consumer Engagement on Social Media." *Journal of Interactive Marketing* 53: 32–46. <https://doi.org/10.1016/j.intmar.2020.06.005>.
- Rashkin, Hannah, Eunsol Choi, Jin Yea Jang, Svitlana Volkova, and Yejin Choi. 2017. "Truth of Varying Shades: Analyzing Language in Fake News and Political Fact-Checking." In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2931–37. Association for Computational Linguistics.
- Shu, Kai, Amy Sliva, Suhan Wang, Jiliang Tang, and Huan Liu. 2017. "Fake News Detection on Social Media: A Data Mining Perspective." *ACM SIGKDD Explorations Newsletter* 19 (1): 22–36.